VCL2FMX Converter V3 Runtime Flow

This document describes the actual runtime flow of the VCL-to-FMX converter as it runs today in the V3 codebase. It is based on the live code path in:

- `C:\New Delphi Projects\VCL2FMXConverterV3\MainForm.pas`
- `C:\New Delphi Projects\VCL2FMXConverterV3\Converter.Core.Engine.pas`
- `C:\New Delphi Projects\VCL2FMXConverterV3\Converter.Core.FileManager.pas`
- `C:\New Delphi Projects\VCL2FMXConverterV3\Converter.Core.Integration.pas`
- `C:\New Delphi Projects\VCL2FMXConverterV3\Converter.Parser.Pascal.pas`
- `C:\New Delphi Projects\VCL2FMXConverterV3\Converter.Parser.DFM.pas`
- `C:\New Delphi Projects\VCL2FMXConverterV3\Converter.Project.Generator.pas`

This is not a marketing description. It is the operational flow from program startup through conversion, file generation, report generation, and post-run UI state.

1. Startup Sequence

When the program launches, the main form is created first. The first major runtime event is `TfrmMain.FormCreate`.

`FormCreate` does all of the initial bootstrapping for the user interface and the converter session state.

It explicitly wires runtime event handlers:

1. `OnCloseQuery := FormCloseQuery`
2. `OnShow := FormShow`

3. `OnResize := FormResize`
4. `TabControlMain.OnChange := TabControlMainChange`

It then initializes the file-type selector:

5. clears the combo box
6. adds `PAS files only`
7. adds `DFM files only`
8. adds `Both PAS and DFM`
9. sets the default selection to `Both PAS and DFM`

It clears the source and target folder edits:

10. `edtSource.Text := ''`
11. `edtTarget.Text := ''`

It enables the main conversion option checkboxes by default:

12. recursive scanning = on
13. critical areas = on
14. data-aware conversion = on
15. third-party handling = on
16. WinAPI handling = on

It seeds the live log with the startup instruction text:

`Choose a VCL source folder and an FMX output folder, then start the conversion.`

It makes the built-in reference memos read-only, then generates and loads the reference-page content for:

17. component mappings
18. property mappings
19. event mappings

It then resets all run artifacts and summary fields by calling `ResetArtifacts`, updates the rule summary, marks the UI as not currently converting with `UpdateUIForConversion(False)`, builds the tab chrome, sets the active tab to `tabProjectScan`, updates the tab chrome again, and applies the responsive layout.

That means that by the time `FormCreate` finishes, the converter is already in a clean "ready for first use" state.

2. What `FormShow` Does

After creation, FMX raises `FormShow`.

`FormShow` does additional startup work that is display-related rather than data-entry-related:

20. marks the form as shown
21. maximizes the window
22. applies responsive layout again
23. applies the tab theme
24. updates the tab chrome
25. ensures the embedded web-browser controls exist for the reference pages
26. reloads the component, property, and event reference content into those viewers

One important detail:

`FormCreate` already set the active tab to `tabProjectScan`, so the `FormShow` fallback that would choose `tabDashboard` usually does not change anything. In normal use, the application opens on the Project Scan tab, not the Dashboard tab.

At that point, the form is visible and waits for user activity.

3. Idle Ready State After Startup

Once startup is complete, the program is effectively sitting in a ready state.

The converter is not yet running.

The UI state at this point is:

27. source folder is blank
28. target folder is blank
29. convert options are all on by default
30. file-type filter is `Both PAS and DFM`
31. status text says the converter is ready
32. no report path exists yet
33. no output folder path exists yet
34. open-report, print-report, and open-output actions are disabled because there is nothing to open yet

4. Source Folder Selection Flow

When the user clicks `Browse Source`, `btnBrowseSourceClick` runs.

This handler:

35. copies the current `edtSource.Text` into a local `Dir` variable
36. opens `SelectDirectory('Select Source Folder', '', Dir)`
37. if the user confirms a folder, it assigns that folder back into `edtSource.Text`
38. it then calls `SuggestTargetFromSource(Dir)`

`SuggestTargetFromSource` does not create a folder. It only proposes a default target path in the target edit box.

It works like this:

39. if `edtTarget` already contains text, it does nothing
40. it strips any trailing delimiter from the source path
41. it gets the source folder's parent directory
42. it gets the source folder name itself
43. it builds:

`<ParentFolder>\<SourceFolderName> - FMX Output`

So if the source is:

`C:\New Delphi Projects\Carillon7_2_4 - Portable`

the suggested target becomes:

`C:\New Delphi Projects\Carillon7_2_4 - Portable - FMX Output`

Important detail:

The target folder is only suggested here. It is not created yet.

5. Target Folder Selection Flow

When the user clicks `Browse Target`, `btnBrowseTargetClick` runs.

This handler:

44. copies `edtTarget.Text` into a local `Dir` variable
45. opens `SelectDirectory('Select Target Folder', '', Dir)`
46. if the user confirms a folder, it writes that folder into `edtTarget.Text`

Again, this does not create the folder yet. It only captures the intended output path.

6. What Happens When the User Clicks Convert

When the user clicks `Convert Project`, `btnConvertClick` runs.

This is the handoff point from the user-facing UI into the real conversion pipeline.

The convert handler performs a strict set of validations before it starts any worker thread.

6.1. It first blocks duplicate runs

If `FConversionThread` is already assigned, it shows:

`A conversion is already running.`

and exits immediately.

6.2. It reads and trims the source and target edits

It stores:

47. `Src := edtSource.Text.Trim`
48. `Dst := edtTarget.Text.Trim`

6.3. It validates the source and target paths

It stops and shows a message if:

49. source folder is blank
50. target folder is blank
51. source folder does not exist
52. source folder and target folder resolve to the same actual path

If source and target are the same, it shows:

`Please choose a different output folder. The source and target folders should not be the same.`

It also switches the tab back to `tabProjectScan` when path validation fails, so the operator returns to the folder-selection screen.

6.4. It resets the visible run state

Once validation passes, it:

53. clears the log
54. adds `Preparing conversion...`
55. switches the active tab to `tabIssues`

That means the user is moved from the selection screen to the live run screen immediately before the worker thread starts.

6.5. It captures the current conversion options

It reads:

56. selected file type
57. recursive on/off
58. critical-areas on/off
59. data-aware on/off
60. third-party on/off
61. WinAPI on/off

6.6. It primes the run-summary fields

Before the thread starts, it resets all pending counters and display fields:

62. last output path becomes the chosen target path
63. last report paths are cleared

64. pending files processed = 0
65. pending files converted = 0
66. pending files with errors = 0
67. pending files skipped = 0
68. pending manual review count = 0
69. pending blocking count = 0
70. pending duration text = empty

It then updates the dashboard and issues-tab labels so the operator can already see where output and report files are expected to land.

It also calls `UpdateConversionOutputSummary(True)`, which changes the summary text to:

`Conversion is running. The live log below is updating in real time, and run totals will appear here when the pass finishes.`

6.7. It disables the interactive controls

`UpdateUIForConversion(True)` disables:

71. Convert
72. Reset
73. Browse Source
74. Browse Target
75. file-type combo
76. all main option checkboxes

It also:

77. changes the convert button text to `Converting...`

78. sets status text to `Running conversion`
79. disables open-report
80. disables print-report
81. disables open-output

6.8. It creates the worker thread

The actual conversion work does not happen on the UI thread.

`btnConvertClick` creates an anonymous thread and starts it. That thread calls:

`ConversionThreadProc(Src, Dst, FileTypeIndex, Recursive, EnableCritical, EnableDataAware, EnableThirdParty, EnableWinAPI)`

Inside the thread wrapper:

82. `StatusText` starts as `Conversion failed`
83. the thread runs `ConversionThreadProc`
84. regardless of success or failure, it stores:
    - `FPendingCompletionTargetPath := Dst`
    - `FPendingCompletionStatusText := StatusText`
85. it then queues `CompleteConversionOnUIThread` back to the main thread using `TThread.Queue`

So the heavy work is done in the background, but the final UI update is always returned safely to the main thread.

7. What `ConversionThreadProc` Does

`ConversionThreadProc` is the thread-side entrypoint for the conversion session.

It creates two major objects:

86. `TConversionContext`
87. `TConverterEngine`

7.1. It builds the conversion context

The context receives the selected run options:

88. `SourcePath`
89. `OutputPath`
90. `ProcessSubdirectories`
91. `CreateReport := True`
92. `EnableCriticalAreas`
93. `EnableDataAware`
94. `EnableThirdParty`
95. `EnableWinAPI`
96. file-type filter:

- `ftPas`
- `ftDfm`
- or `ftBoth`

7.2. It logs that the engine is starting

It sends:

`Starting conversion engine...`

to the log.

7.3. It creates the engine and runs it

It instantiates `TConverterEngine`, assigns the screen memo for live log updates, and calls:

`Engine.Convert(Context)`

7.4. It captures post-run totals

After `Engine.Convert` returns, it copies the final counts from the engine and context into the formâ€™s pending summary fields:

97. files processed
98. files converted
99. files with errors
100. files skipped
101. manual review count
102. blocking count
103. formatted duration text

7.5. It decides the final status text

The final run status shown back in the UI is decided like this:

104. if the engine recorded file errors, or the context contains blocking issues:

`Blocking issues present`

105. else if the context contains manual-review issues:

`Manual review required`

106. else:

`Clean conversion`

If an exception escapes the thread procedure, it shows an error through the synchronized UI path and leaves the final status as:

`Conversion failed`

8. What the Engine Builds Internally Before the Run

`TConverterEngine.Create` constructs the three main subsystems used during a run:

107. `TFileManager`
108. `TConversionOrchestrator`
109. `TProjectGenerator`

Those three objects are the core of the conversion pipeline.

In practical terms:

110. the file manager decides what source files are eligible and where output files are saved
111. the orchestrator converts Pascal and DFM content
112. the project generator creates the output-side project files and copies/stages companion assets

9. Engine Run Flow: `TConverterEngine.Convert`

The engineâ€™s `Convert` method is the central pipeline controller.

9.1. It resets internal run state

At the beginning of every run it clears:

113. cancellation flag
114. processed count
115. converted count
116. error count
117. skipped count
118. start time
119. context issues
120. internal conversion log

It also records the start time into the conversion context.

9.2. It initializes live logging

If no screen memo is attached, it logs that UI updates are disabled.

Normally, because the main form assigned `Engine.ScreenMemo := MemoLog`, the live log is active.

It then writes the conversion header:

121. blank line
122. `VCL TO FMX CONVERSION STARTED`
123. separator line
124. source path
125. output path

9.3. It resets and prepares the file manager

The engine calls:

126. `FFileManager.Reset`
127. `FFileManager.PrepareOutput`

These are not the same thing.

`Reset` scans the source side and builds the list of candidate files.

`PrepareOutput` prepares the target side.

10. Source Scan Rules: What Files Are Collected

The file manager starts by clearing its internal file list and resetting its index.

It then verifies that the source directory exists. If it does not, it adds an error issue and stops.

10.1. The output folder is excluded from scanning

If the output folder sits under the source tree, the file manager deliberately skips it so the converter does not re-read its own generated output as input.

10.2. Excluded folders

The file manager excludes any path segment matching:

128. `backup`
129. `backup copy`
130. `win32`
131. `win64`
132. `deploy`
133. `__history`
134. `.git`
135. `.svn`
136. `recovery_generic`
137. `debug`
138. `release`

10.3. Recursive vs top-only scan

If recursive mode is on, subdirectories are scanned.

If recursive mode is off, only the top directory is scanned.

10.4. File-type filtering

After a file is found, it must match the currently selected file type:

139. `.pas` only
140. `.dfm` only
141. both `.pas` and `.dfm`

10.5. File order

After collection, the file list is sorted.

That means the converter processes files in sorted order, not raw filesystem discovery order.

11. Output Preparation Rules

`PrepareOutput` is where the converter actually makes the output directory real if it does not exist already.

This is the point where the target folder can be created.

It does the following:

142. validates that an output path exists in the options
143. if the output folder does not exist, calls `ForceDirectories`
144. removes stale build-artifact folders from the output path if they exist:

- `Win32`
- `Win64`
- `Debug`
- `Release`

- `__history`

So the output path is not just created. It is also cleaned of the most common stale build subfolders before file conversion begins.

12. File-Processing Loop

After scan and output preparation, the engine asks the file manager for the file count and logs:

`Found X files to process`

It then loops while:

145. more files exist
146. the run has not been cancelled

For each file it:

147. gets the next full file path
148. increments the processed counter
149. logs `[n/total] Processing: filename`
150. routes the file by extension:

- `.pas` -> `ProcessPasFile`
- `.dfm` -> `ProcessDfmFile`

If an exception occurs around a file-level pass, the engine:

151. increments file-error count
152. logs a failed message
153. adds a blocking issue to the context

13. Pascal File Flow

When a Pascal file is processed, this is the actual sequence.

13.1. The file is read

The engine:

154. logs that it is processing the Pascal file
155. sets `FContext.CurrentFile`
156. tries to read the file using encoding detection and fallback logic

If the read fails:

157. the file is counted as skipped
158. a failure is logged
159. no conversion is attempted

13.2. The orchestrator converts the Pascal code

If reading succeeds, the engine calls:

`FOrchestrator.ConvertPascal(AFileName, Code)`

Inside the orchestrator, the Pascal pass runs in this order:

160. add informational issue: `Converting Pascal: <file>`
161. prepare form data-binding metadata
162. align shape field types
163. rewrite folder-picker dialogs
164. rewrite simple grid-draw handlers
165. deep-parse the Pascal unit with `TPascalParser`
166. process method bodies for message, GDI, synchronize, and similar patterns
167. apply automatic text fixes
168. keep structural wrapper panels synchronized with DFM-side layout mapping
169. apply advanced converters
170. run automatic fixes again to sanitize code introduced by advanced converters
171. wrap bitmap canvas drawing in FMX `BeginScene` / `EndScene`
172. normalize stored VCL runtime colors for FMX usage
173. promote full-text memo fields
174. inject LiveBindings for converted data-aware controls
175. reduce generated cleanup noise
176. inject compatibility classes where needed
177. inject runtime lifecycle fixes
178. repair the implementation section
179. remove duplicate implementation uses clauses
180. rewrite root-form double-click handling
181. rewrite runtime text-layout math
182. perform final uses-clause cleanup
183. add informational issue: `Pascal conversion successful: <file>`

That is the real Pascal conversion chain in the live V3 code.

13.3. The converted Pascal file is saved

If conversion succeeds:

184. the engine increments `FilesConverted`
185. the file manager computes the output file name using the path relative to the source root
186. it creates any needed subdirectories in the output tree
187. it sanitizes invalid control characters from the output text
188. it writes the file while preserving the source encoding style as closely as possible
189. it adds an informational issue describing what was saved

If conversion fails:

190. the engine increments file errors
191. logs the failure
192. adds a warning or error issue to the context
193. does not save the converted file

14. DFM File Flow

When a DFM file is processed, this is the actual sequence.

14.1. The DFM is loaded

The engine logs that it is processing the DFM and sets `FContext.CurrentFile`.

Inside the orchestrator, the DFM parser loads the file through `LoadDFM`.

If the DFM is binary:

194. it detects the `TPF0` signature
195. converts binary DFM to text using `ObjectBinaryToText`

If the DFM is already text:

196. it tries UTF-8
197. falls back to ANSI if needed

14.2. The DFM text is parsed into a component tree

The parser:

198. clears the old component list
199. resets the current component and collection state
200. walks the DFM line by line
201. parses `object` / `inherited` declarations
202. parses collections and subcomponents
203. parses properties and events
204. records original line numbers for properties and events

The result is an internal tree of `TDFMComponent` objects describing the source form.

14.3. Component analysis happens before FMX output is generated

The orchestrator then:

205. recursively analyzes components for third-party detection if third-party handling is enabled
206. runs data-aware analysis if data-aware handling is enabled
207. clears old data-binding metadata if data-aware handling is disabled

14.4. The FMX form text is generated

The parser then converts the parsed DFM tree into FMX text by calling:

`FDfmParser.Convert(FComponentMapper)`

The FMX generator:

208. takes the parsed root component
209. generates the FMX object tree
210. uses the component mapper to translate VCL classes, properties, and events
211. normalizes generated FMX values such as align constants
212. emits FMX text for the output form

After that, the orchestrator applies the full-text memo promotion pass to the generated FMX text and replaces the original DFM text with the generated FMX version.

It then logs:

`DFM conversion successful: <file>`

14.5. The output filename is changed from `.dfm` to `.fmx`

When the file manager saves a converted DFM, it automatically changes the output extension from `.dfm` to `.fmx`.

That means:

`SomeForm.dfm`

becomes:

`SomeForm.fmx`

in the FMX output tree.

15. What Happens After All Source Files Are Converted

After the file loop finishes, the engine logs:

`Generating project files...`

It then runs:

213. `FProjectGenerator.GenerateProject`
214. `AuditGeneratedOutput`
215. `GenerateReport`

in that order.

16. Project Generation Flow

`TProjectGenerator.GenerateProject` performs four major actions:

216. `CopyDPR`
217. `CopyDPROJ`
218. `CopyDeployProj`
219. `CopyProjectCcompanionFiles`

It then adds an informational issue:

`Delphi project files generated for FMX output`

16.1. DPR generation

The generator:

220. finds the original `.dpr`
221. reads it
222. transforms the startup content
223. writes the transformed `.dpr` into the FMX output folder as UTF-8

16.2. DPROJ generation

The generator:

224. finds the original `.dproj`
225. reads it
226. transforms project metadata and namespaces
227. writes the transformed `.dproj` into the FMX output folder as UTF-8

16.3. DEPLOYPROJ generation

If a `.deployproj` exists, it is likewise found, transformed, and written into the output folder.

16.4. Companion-file and runtime-support pass

The project generator then:

228. copies root-level project companion files such as `.res`, `.ico`, `.manifest`, `.png`, and `.bmp`
229. inspects project metadata for additional referenced assets
230. stages referenced assets into the output tree when possible
231. records warnings when referenced assets cannot be copied
232. scans the source tree for runtime-support files such as `.exe`, `.dll`, `.lib`, and `.sf2`
233. identifies stageable runtime companions such as `.exe`, `.dll`, and `.sf2`
234. stages selected runtime companions into the FMX output directory
235. records informational notes in the report about what was staged and what still requires deployment review

17. Output Audit Pass

After project generation, the engine audits the generated output.

This pass walks the output tree looking at:

236. `.pas`
237. `.dpr`
238. `.fmx`

Important detail:

It only audits files touched during the current run. That is done by comparing file write times against the run start time. This is specifically meant to avoid stale leftovers in reused output folders from polluting the report for the current conversion.

18. Report Generation Flow

If report creation is enabled, the engine generates both:

239. `VCL_to_FMX_Conversion_Report.txt`
240. `VCL_to_FMX_Conversion_Report.html`

Both are written directly into the output folder root.

18.1. The report calculates counts and status

The report pass calculates:

241. informational issue count
242. warning count
243. manual-review group count
244. error count
245. blocking-item count
246. total visible issue count
247. distinct files needing attention

18.2. The final status is assigned

The final status text inside the report is:

248. `Blocking issues present` if there were file errors or blocking issues
249. `Manual review required` if there were manual-review items but no blocking outcome
250. `Clean conversion` otherwise

18.3. The text report contents

The text report includes:

251. conversion summary
252. total time
253. files processed
254. files converted
255. files with errors
256. files skipped
257. total issues
258. distinct files needing attention
259. final status
260. manual fixes required
261. blocking items
262. files with conversion errors
263. recommended next step
264. detailed issues
265. grouped manual-review section
266. issue summary table

18.4. The HTML report contents

The HTML report contains the same logical data in a styled HTML dashboard format, including:

267. hero summary/status area
268. metric cards
269. detailed issue cards

270. grouped manual-review cards
271. issue summary table
272. recommended next-step guidance

19. Engine Completion Logic

After the report is written, the engine decides how to log completion.

If blocking issues exist:

273. it logs a message saying blocking issues are present
274. it returns `False`

If there are no blocking issues but manual-review issues exist:

275. it logs that manual review is required
276. it returns `True`

If neither blocking nor manual-review issues exist:

277. it logs that conversion completed cleanly
278. it returns `True`

Important detail:

The worker-thread status string shown back in the main form is decided separately in `ConversionThreadProc`, but it uses the same basic logic.

20. How the UI Is Restored After the Worker Thread Finishes

When the background thread ends, it does not directly update the UI.

Instead it queues `CompleteConversionOnUIThread`.

That method runs back on the main thread and does exactly three things:

279. frees the conversion thread object
280. calls `UpdateUIForConversion(False)` to re-enable controls
281. calls `UpdateArtifactsAfterConversion(TargetPath, StatusText)`

21. What `UpdateArtifactsAfterConversion` Does

This is the method that makes the finished run visible to the operator.

It:

282. marks that a run has completed
283. copies all pending counters into the last-run fields
284. stores the last output path
285. computes the expected text and HTML report paths
286. picks the preferred report path, preferring HTML over text
287. updates the status labels
288. updates dashboard output and report labels
289. updates issues-tab output and report labels
290. rebuilds the summary sentence
291. re-enables open-report, print-report, and open-output actions as appropriate

So this is the moment when the UI leaves the "running" state and becomes a post-run review screen.

22. What the User Can Do After the Run

Once conversion is complete and the UI is unlocked, the operator can:

292. open the output folder
293. open the preferred report
294. print the preferred report
295. reset the converter back to its startup state

22.1. Open Output

`btnOpenOutputClick`:

296. checks whether `FLastOutputPath` exists
297. if not, shows `There is no converted output folder to open yet.`
298. otherwise uses Windows `ShellExecute('open', path)` to open the folder

22.2. Open Report

`btnOpenReportClick`:

299. asks for the preferred report path
300. prefers HTML if it exists
301. otherwise uses the text report if it exists
302. if neither exists, shows `There is no generated report to open yet.`
303. otherwise opens the report with `ShellExecute('open', path)`

22.3. Print Report

`btnPrintReportClick`:

304. resolves the preferred report path
305. if none exists, shows `There is no generated report to print yet.`
306. attempts `ShellExecute('print', reportPath, hideWindow=True)`
307. if direct print is not available, it opens the report instead and tells the user that direct printing was not available

23. Reset Flow

If the user clicks `Reset`, `btnResetClick` runs.

If a conversion thread is still active, reset is blocked with:

`Please wait for the current conversion to finish before resetting.`

If no conversion is running, `ResetToStartupState` is called.

That method:

308. clears source and target edits
309. restores file type to `Both PAS and DFM`
310. re-enables all main option checkboxes to their default checked state
311. clears pending completion and message fields
312. restores the startup log text
313. rebuilds and reloads the reference-page content

314. resets artifacts and run summary
315. updates the rule summary
316. marks the UI as not converting
317. scrolls the tab scroll boxes back to the top
318. returns the active tab to `tabProjectScan`
319. applies responsive layout
320. moves focus back to the source edit if possible

So reset is a full UI-state reset, not just a log clear.

24. Close Protection While a Run Is Active

If the user tries to close the program while a conversion is still running, `FormCloseQuery` blocks the close.

It only allows the program to close when `FConversionThread` is not assigned.

If a run is active, it shows:

`A conversion is still running. Please wait for it to finish before closing the program.`

That means the converter deliberately prevents shutdown in the middle of an active conversion pass.

25. Thread Boundary Summary

There are two major runtime domains in the program:

25.1. UI thread

The UI thread handles:

321. form startup
322. folder selection
323. validation
324. starting the worker thread
325. log display updates
326. synchronized error dialogs
327. final completion UI updates
328. post-run actions like open-report, print-report, open-output, and reset

25.2. Worker thread

The worker thread handles:

329. conversion context creation
330. engine construction
331. source scan
332. output preparation
333. per-file Pascal conversion
334. per-file DFM conversion
335. project generation
336. generated-output audit
337. report generation
338. final run-status determination

The worker thread never directly finalizes the UI. It hands completion back through `TThread.Queue`.

26. Exact Start-to-Finish Narrative in Plain English

If you want the whole thing in one straight-through human narrative, this is it:

339. The program starts.
340. `FormCreate` runs.
341. The form wires its runtime events, loads the file-type options, defaults all major conversion modules to on, loads the mapping-reference material, resets all run artifacts, and sets the active tab to Project Scan.
342. `FormShow` runs.
343. The window maximizes, the theme/layout pass runs, and the embedded reference browsers are prepared.
344. The form is displayed and waits for user activity.
345. The user chooses a source folder.
346. The converter writes that folder into `edtSource`.
347. If the target folder is still blank, the converter suggests a sibling folder named `<SourceName> - FMX Output`.
348. The user can accept that target or browse for another target folder.
349. Nothing is created yet just from selecting folders.
350. The user clicks `Convert Project`.
351. The converter validates source and target.
352. It rejects blank paths, missing source folders, or a source path equal to the target path.
353. If validation succeeds, it clears the log, switches to the Issues tab, resets pending counters, updates output/report labels, disables interactive controls, changes the button text to `Converting...`, and starts a worker thread.
354. The worker thread creates a conversion context and copies in all selected options.
355. The worker thread creates the engine.
356. The engine resets its counters and issues and logs the start banner.
357. The file manager scans the source tree, skipping excluded directories and skipping the output tree if it sits under the source tree.
358. The output directory is created if needed, and stale build subfolders under the output folder are deleted.
359. The engine loops through the sorted list of eligible `.pas` and `.dfm` files.
360. Each Pascal file is read, parsed, transformed through the orchestratorâ€™s multi-step pipeline, and saved into the output tree in relative position.

361. Each DFM file is loaded, converted from binary to text if needed, parsed into components, translated into FMX text, renamed to `.fmx`, and saved into the output tree in relative position.
362. After source files finish, the project generator creates transformed `.dpr`, `.dproj`, and `.deployproj` files and copies or stages project assets and runtime companion files.
363. The engine audits the generated output that was touched during the current run.
364. The engine writes both the text and HTML conversion reports into the output folder.
365. The engine decides whether the run was clean, manual-review, or blocking.
366. The worker thread stores the final status and queues the completion handler back to the main thread.
367. The UI thread receives that completion callback.
368. The UI re-enables all controls, stores the final output and report paths, updates the dashboard and issues panels, and enables the report/output actions.
369. The user can now open the output folder, open the HTML report, print the report, inspect the generated FMX project, or reset the converter for another run.

27. Bottom Line

At a practical level, the converter is a two-stage system:

370. a UI/controller stage in `MainForm.pas`
371. a background conversion pipeline stage driven by `TConverterEngine`

The user-facing screen never directly performs the actual conversion. It validates inputs, starts the worker thread, and then later publishes the result of that thread back into the UI.

The actual conversion pipeline is:

372. collect options
373. scan eligible files
374. prepare output
375. convert Pascal

376. convert DFM to FMX
377. generate project files
378. copy/stage assets and runtime companions
379. audit generated output
380. generate reports
381. return status and counts to the UI

That is the complete operational flow of the converter from startup to completed conversion.